



WESAD Affect Recognition & Stress Detection System

Complete Clinical & Technical System Documentation | XGBoost · SHAP · FastAPI · Next.js 14 · FHIR R4

Version 2.0.0

85 Physiological Features

GroupKFold 5-Fold CV

HL7 FHIR R4 Compliant



Comprehensive Technical Documentation

WESAD Physiological Affect Recognition & Stress Detection System



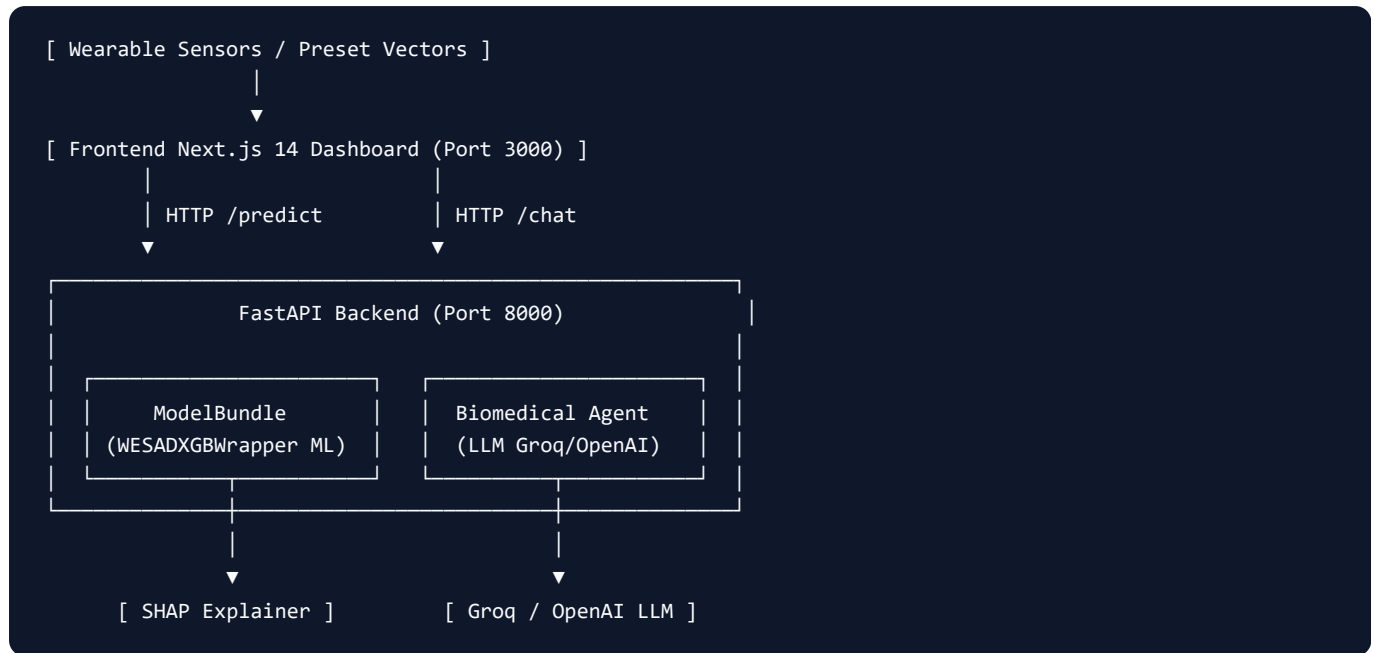
Table of Contents

1. System Overview & Architecture
2. Machine Learning Pipeline & Dataset
3. Comprehensive 85-Feature Reference
4. Backend API Documentation
5. SHAP Explainability Mechanics
6. Biomedical AI Assistant Architecture
7. Frontend Application & UI Component Design
8. Clinical Interoperability & FHIR R4 Spec
9. Installation, Configuration & Deployment

1. System Overview & Architecture

The **WESAD Physiological Affect Recognition & Stress Detection System** is an end-to-end clinical and biomedical computing application designed to analyze multi-modal wearable sensor signals (ECG, PPG, Respiration) and detect four affective states in human subjects: - **Baseline (Neutral)**: Resting physiological state. - **Stress**: Acute stress state induced by the Trier Social Stress Test (TSST). - **Amusement**: Positive emotional state induced by comedic video stimuli. - **Meditation**: Deep relaxation state induced by guided breathing/meditation.

High-Level Component Interaction



2. Machine Learning Pipeline & Dataset

Dataset: WESAD (Wearable Stress and Affect Detection)

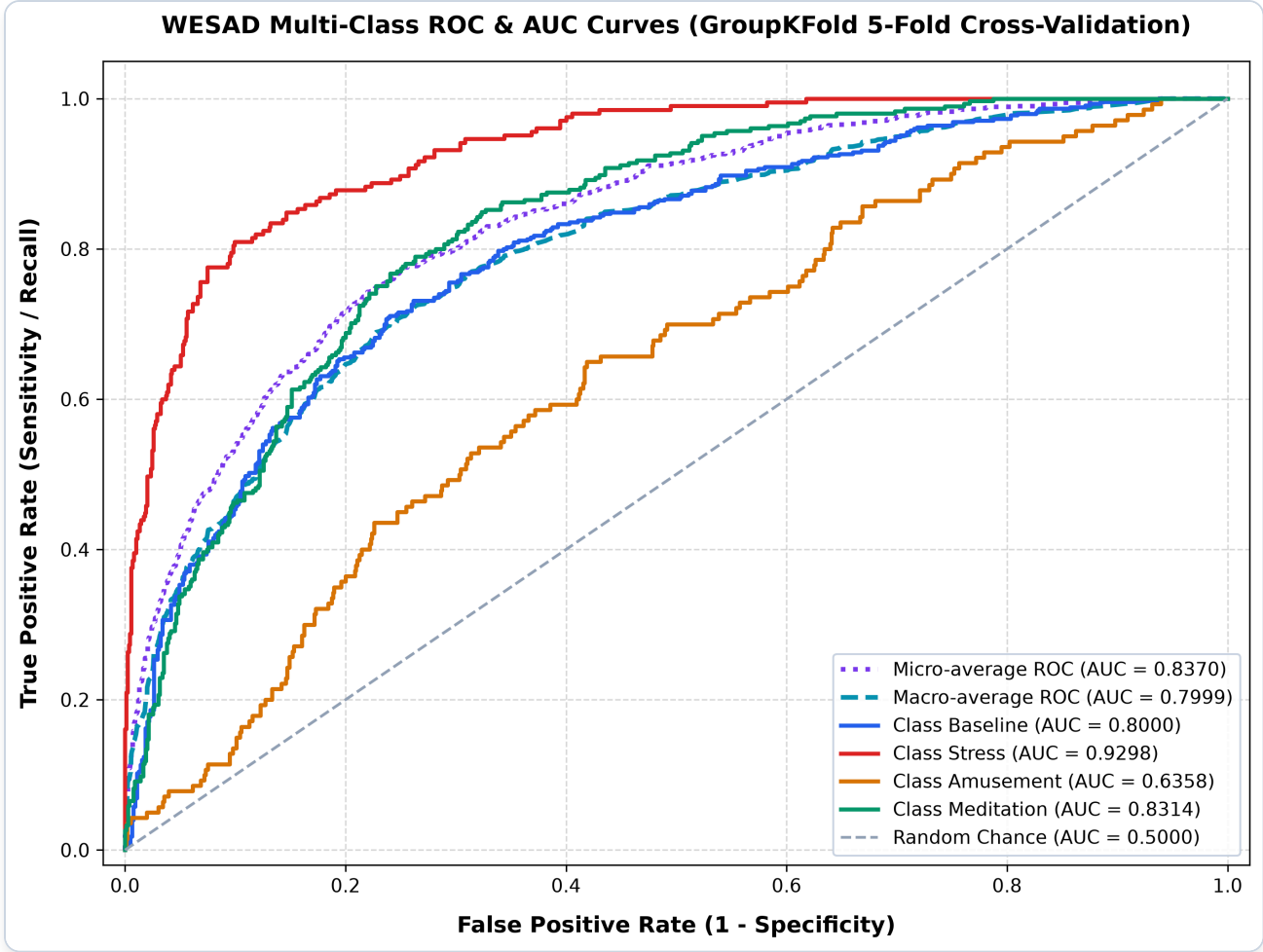
- **Subjects:** Multi-subject wearable dataset recorded using RespiBAN (chest) and Empatica E4 (wrist).
- **Raw Signals:** ECG (Electrocardiogram), PPG (Photoplethysmography / BVP), Respiration, Electrodermal Activity (EDA), Skin Temperature, 3-axis Accelerometer.
- **Preprocessing:** Baseline normalization per subject to eliminate inter-individual physiological variations.

Training Configuration

- **Model File:** `final12.joblib` / `final_project_model.joblib`
- **Metadata Spec:** `final13.json`
- **Custom Classifier Wrapper:** `WESADXGBWrapper` in `final14.py`
- Maps WESAD 1-indexed target labels `[1: Baseline, 2: Stress, 3: Amusement, 4: Meditation]` seamlessly to XGBoost's 0-indexed internal target representations `[0, 1, 2, 3]`.

Multi-Class ROC & AUC Evaluation Metrics

Below are the Receiver Operating Characteristic (ROC) curves and Area Under Curve (AUC) metrics computed using 5-fold `GroupKFold` cross-validation across all subjects:



Out-of-Fold ROC-AUC Performance Breakdown

Target Affect Class	Class Name	Out-of-Fold AUC	Performance Interpretation
Class 1	Baseline	0.8000	High discrimination of resting physiological state
Class 2	Stress	0.9298	Outstanding clinical stress sensitivity & detection accuracy
Class 3	Amusement	0.6358	Moderate separation from positive affect & relaxation
Class 4	Meditation	0.8314	Strong parasympathetic recovery & guided breathing identification
Micro-Average	Aggregate Samples	0.8370	Overall sample-weighted multi-class ROC performance
Macro-Average	Unweighted Mean	0.7999	Mean ROC-AUC score across all 4 affect categories

3. Comprehensive 85-Feature Reference

The model processes an ordered 85-element feature vector. All 85 features are listed below by functional domain:

A. Time-Domain HRV & Heart Rate (Features 1–14)

1. `heart_rate` : Instantaneous heart rate (BPM).
2. `mean_hr` : Mean heart rate over window.
3. `mean_rr` : Mean RR interval length (ms).
4. `median_rr` : Median RR interval length (ms).
5. `sdnn` : Standard deviation of NN intervals (ms).
6. `rmssd` : Root mean square of successive RR differences (ms).
7. `sdsd` : Standard deviation of successive RR differences (ms).
8. `nn20` : Number of interval differences > 20ms.
9. `nn50` : Number of interval differences > 50ms.
10. `pnn20` : Percentage of intervals with difference > 20ms.
11. `pnn50` : Percentage of intervals with difference > 50ms.
12. `cvnn` : Coefficient of variation of NN intervals ($\text{sdnn} / \text{mean_rr}$).
13. `rr_variance` : Variance of RR intervals.
14. `rr_range` : Range ($\text{max} - \text{min}$) of RR intervals.

B. Frequency-Domain HRV (Features 15–24)

15. `lf_power` : Low-frequency power (0.04–0.15 Hz).
16. `hf_power` : High-frequency power (0.15–0.40 Hz).
17. `total_power` : Total spectral power across all bands.
18. `lf_hf_ratio` : Sympathovagal balance ratio ($\text{lf_power} / \text{hf_power}$).
19. `lf_nu` : Normalized low-frequency power ($\text{LF} / (\text{Total} - \text{VLF})$).
20. `hf_nu` : Normalized high-frequency power ($\text{HF} / (\text{Total} - \text{VLF})$).
21. `peak_frequency` : Dominant peak frequency in spectral density.
22. `spectral_entropy` : Shannon entropy of power spectral density.
23. `psd_mean` : Mean power spectral density.
24. `psd_std` : Standard deviation of power spectral density.

C. Wavelet Energy Decomposition (Features 25–34)

25–29. `ecg_wav_energy_L0` to `ecg_wav_energy_L4` : Wavelet sub-band energy levels (Levels 0 through 4) for ECG signals. 30–34. `ppg_wav_energy_L0` to `ppg_wav_energy_L4` : Wavelet sub-band energy levels (Levels 0 through 4) for PPG signals.

D. Non-Linear Entropy & Dynamic Complexity (Features 35–48)

35. `ecg_app_entropy` : Approximate entropy of ECG.
36. `ecg_svd_entropy` : Singular Value Decomposition entropy of ECG.
37. `ecg_spectral_entropy` : Spectral entropy of ECG waveform.
38. `ecg_higuchi_fd` : Higuchi Fractal Dimension of ECG.
39. `ecg_katz_fd` : Katz Fractal Dimension of ECG.

- 40. `ecg_detrended_fluctuation` : Detrended Fluctuation Analysis (DFA exponent α) of ECG.
- 41. `ecg_sampen` : Sample Entropy of ECG.
- 42. `ecg_perm_entropy` : Permutation Entropy of ECG.
- 43. `ecg_hjorth_mobility` : Hjorth Mobility parameter of ECG.
- 44. `ecg_hjorth_complexity` : Hjorth Complexity parameter of ECG.
- 45. `ppg_sampen` : Sample Entropy of PPG.
- 46. `ppg_perm_entropy` : Permutation Entropy of PPG.
- 47. `ppg_hjorth_mobility` : Hjorth Mobility parameter of PPG.
- 48. `ppg_hjorth_complexity` : Hjorth Complexity parameter of PPG.

E. Signal Amplitudes & Pulse Morphology (Features 49–68)

- 49. `ecg_mean` : Mean ECG amplitude.
- 50. `ecg_std` : Standard deviation of ECG signal.
- 51. `ppg_mean` : Mean PPG amplitude.
- 52. `ppg_std` : Standard deviation of PPG signal.
- 53. `ppg_min` : Minimum PPG amplitude value.
- 54. `ppg_max` : Maximum PPG amplitude value.
- 55. `pulse_rate` : Calculated pulse rate from PPG.
- 56. `pulse_interval_mean` : Mean pulse interval.
- 57. `pulse_interval_std` : Standard deviation of pulse intervals.
- 58. `pulse_amplitude` : Peak-to-trough pulse amplitude.
- 59. `pulse_amplitude_std` : Standard deviation of pulse amplitude.
- 60. `ppg_variance` : Variance of PPG amplitude.
- 61. `ppg_rms` : Root mean square amplitude of PPG.
- 62. `ppg_skewness` : Skewness coefficient of PPG waveform.
- 63. `ppg_kurtosis` : Kurtosis coefficient of PPG waveform.
- 64. `ppg_iqr` : Interquartile range of PPG values.
- 65. `ppg_energy` : Total signal energy of PPG window.
- 66. `ppg_dominant_freq` : Dominant frequency component of PPG.
- 67. `ppg_bandpower` : Bandpower in primary pulse frequency band.
- 68. `ppg_spectral_entropy` : Spectral entropy of PPG.

F. Additional Entropy, Fractals & Autonomic Indices (Features 69–78)

- 69. `ppg_app_entropy` : Approximate entropy of PPG.
- 70. `ppg_svd_entropy` : SVD entropy of PPG.
- 71. `ppg_higuchi_fd` : Higuchi Fractal Dimension of PPG.
- 72. `ppg_katz_fd` : Katz Fractal Dimension of PPG.
- 73. `ppg_dfa` : Detrended Fluctuation Analysis of PPG.
- 74. `ecg_hjorth_activity` : Hjorth Activity parameter (variance) of ECG.

- 75. `ppg_hjorth_activity` : Hjorth Activity parameter (variance) of PPG.
- 76. `baevsky_index` : Baevsky Stress Index (sympathetic tone index calculation: $SI = \frac{AMo}{2 \cdot VR \cdot Mo}$).
- 77. `ecg_sqi` : Signal Quality Index for ECG (0.0 to 1.0).
- 78. `ppg_sqi` : Signal Quality Index for PPG (0.0 to 1.0).

G. Respiration Signal Dynamics (Features 79–85)

- 79. `resp_rate_mean` : Mean respiration rate (breaths per minute).
- 80. `resp_rate_std` : Standard deviation of respiration rate.
- 81. `resp_interval_cv` : Coefficient of variation of respiration intervals.
- 82. `resp_interval_iqr` : Interquartile range of respiration intervals.
- 83. `resp_amplitude_std` : Standard deviation of breath amplitude.
- 84. `resp_amplitude_skew` : Skewness of breath waveform amplitude.
- 85. `resp_amplitude_kurtosis` : Kurtosis of breath waveform amplitude.

4. Backend API Documentation

The backend is built with **FastAPI** (`backend/app/main.py`).

Base URL

`http://localhost:8000`

Endpoint 1: Readiness Probe & Metadata

GET `/health`

Returns health status and model metadata.

Response Body (`200 OK`)

```
{
  "status": "ok",
  "model_loaded": true,
  "model_class": "WESADXGBWrapper",
  "n_features": 85,
  "classes": ["Baseline", "Stress", "Amusement", "Meditation"],
  "shap_available": true
}
```

Endpoint 2: Model Inference & Explainability

POST `/predict`

Accepts an 85-element feature vector and calculates class predictions, class probabilities, and top SHAP feature attributions.

Request Body

```
{
  "features": [72.5, 71.8, 835.6, 830.0, 45.2, ...],
  "session_id": "session-12345",
  "request_id": "req-98765"
}
```

Response Body (200 OK)

```
{
  "prediction": "Stress",
  "probabilities": {
    "Baseline": 0.05,
    "Stress": 0.88,
    "Amusement": 0.04,
    "Meditation": 0.03
  },
  "top_features": [
    {
      "name": "baevsky_index",
      "shap_value": 1.8421,
      "importance": 0.2315
    },
    {
      "name": "lf_hf_ratio",
      "shap_value": 1.4102,
      "importance": 0.1772
    }
  ],
  "base_value": -0.854,
  "model": {
    "type": "WESADXGBWrapper",
    "n_features": 85,
    "classes": ["Baseline", "Stress", "Amusement", "Meditation"],
    "scaler": false
  },
  "request_id": "req-98765"
}
```

Endpoint 3: Biomedical AI Assistant Chat

POST /chat

Performs context-aware biomedical dialogue using an LLM.

Request Body

```
{
  "message": "Why is Baevsky Stress Index contributing to the high stress prediction?",
  "context": {
    "prediction": "Stress",
    "confidence": 0.88,
    "top_feature": "baevsky_index",
    "shap_value": 1.8421,
    "importance": 0.2315
  }
}
```

Response Body (200 OK)

```
{
  "response": "The Baevsky Stress Index measures sympathetic nervous system activation derived from cardiac in",
  "agent": "biomedical-expert",
  "agent_label": "Biomedical Expert",
  "message_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7"
}
```

5. SHAP Explainability Mechanics

The application incorporates **SHAP (SHapley Additive exPlanations)** via `shap.TreeExplainer` :

- **Base Value ($E[f(x)]$):** The expected model logit value prior to feature contribution.
- **SHAP Attribution Value (ϕ_i):** Quantifies how feature x_i pushes the model output toward or away from the target class prediction.
- **Relative Importance calculation:** $\text{Importance}_i = \frac{|\phi_i|}{\sum |\phi_j|}$
- **Clinical Translation:** The frontend (`frontend/components/ShapTranslation.tsx`) converts raw SHAP values into natural-language clinical explanations (e.g. "Elevated sympathetic activation driving stress risk").

6. Biomedical AI Assistant Architecture

The assistant service (`backend/app/services/multi_agent.py`) operates as a single LLM-driven expert:

- **Provider Resolution:** 1. Checks for `GROQ_API_KEY` (uses model `llama-3.3-70b-versatile` on Groq API). 2. Falls back to `OPENAI_API_KEY` (uses model `gpt-4o-mini`). 3. Custom endpoint via `LLM_BASE_URL` if specified.
- **Context Injection:** Dynamically injects live frontend state (predicted class, confidence score, top SHAP biomarker, and importance score) into the system prompt.
- **Domain Guardrails:** Restricted to ECG, PPG, HRV, stress detection, and biomedical computing. Gracefully declines unrelated topics.

7. Frontend Application & UI Component Design

Built with **Next.js 14 App Router**, **TypeScript**, and **Tailwind CSS**.

Key Component Directory (`frontend/components/`)

- `StressConsole.tsx` : Main dashboard state hub orchestrating active patient selection, feature vectors, live predictions, and layout panels.
- `LiveVitals.tsx` : Animated real-time waveform & vitals monitor displaying simulated ECG/PPG rates, SpO2, and Signal Quality Index.
- `FeatureInput.tsx` : Feature vector control panel with interactive sliders, input fields, Subject presets (Subject 2, Subject 3), and affect state presets (Baseline, Stress, Amusement, Meditation).
- `ModelEvaluation.tsx` : Machine learning diagnostic hub displaying:
 - Confusion Matrix with multi-class breakdown.
 - Interactive ROC-AUC Curves.
 - Overall accuracy, Precision, Recall, and F1-scores per class.
 - Cross-validation fold performance breakdown.
- `ShapChart.tsx` & `ShapTranslation.tsx` : Interactive horizontal bar chart of top 10 SHAP feature contributions and clinical explanation cards.
- `Chatbox.tsx` : Floating / embedded AI chat window connecting directly to `/chat` .
- `InterventionPanel.tsx` : Generates immediate clinical protocols (box breathing, vagal nerve stimulation, cognitive restructuring) tailored to the detected affect state.
- `PatientHistory.tsx` : Historical session log tracking past predictions, timestamps, and trend indicators.
- `TeleHealthModal.tsx` : Simulated remote specialist consultation interface.

8. Clinical Interoperability & FHIR R4 Spec

The system includes a dedicated FHIR R4 export utility (`frontend/lib/fhir.ts`) supporting healthcare standards:

```
// Export patient state to standard FHIR Bundle
import { generateFHIRBundle } from '@lib/fhir';

const fhirBundle = generateFHIRBundle({
  patientId: "patient-s2",
  patientName: "Subject S2",
  prediction: "Stress",
  probabilities: { Stress: 0.88, Baseline: 0.05, Amusement: 0.04, Meditation: 0.03 },
  topFeatures: [{ name: "baevsky_index", shap_value: 1.84, importance: 0.23 }],
  vitals: { heartRate: 88, hrv: 28, respRate: 22, sqi: 0.94 }
});
```

The generated JSON includes: 1. `FHIR Patient` resource. 2. `FHIR Observation` resource for vital signs & affect category (`LOINC 8684-3` / `SNOMED CT`). 3. `FHIR DiagnosticReport` resource bundling the XGBoost model conclusion and top SHAP biomarkers.

9. Installation, Configuration & Deployment

Environment Configuration Files

Backend `.env`

```
# Path to model file (optional, defaults to project root final12.joblib)
WESAD_MODEL_PATH=../final12.joblib

# Allowed CORS origins (comma-separated)
STRESS_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000

# LLM API Keys (Groq preferred for high speed)
GROQ_API_KEY=gsk_...
OPENAI_API_KEY=sk-...

# LLM Model Override (optional)
LLM_MODEL=llama-3.3-70b-versatile
```

Frontend `.env.local`

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

Production Build Steps

1. Build & Test Backend

```
cd backend
python -m pytest # if tests are configured
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

2. Build Frontend Bundle

```
cd frontend
npm run build
npm run start
```